Solution: Merge Sort

Problem Statement

Write Recursive Approach for Merge Sort.

The problem is to implement the Merge Sort algorithm using recursion. Merge Sort is an efficient sorting algorithm that follows the divide-and-conquer approach. It divides the input array into two halves, recursively sorts each half, and then merges the sorted halves to obtain the final sorted array.

Examples

#	Input Array	Output	Explanation
1	[5, 2, 8, 3, 1, 6]	[1, 2, 3, 5, 6, 8]	The input array is divided into two halves: [5, 2, 8] and [3, 1, 6]. Each half is recursively sorted. Then, the sorted halves are merged to obtain the final sorted array.
2	[9, 4, 7, 2, 1]	[1, 2, 4, 7, 9]	The input array is divided into two halves: [9, 4] and [7, 2, 1]. Each half is recursively sorted. Then, the sorted halves are merged to obtain the final sorted array.
3	[3, 1, 2, 5, 4]	[1, 2, 3, 4, 5]	The input array is divided into two halves: [3, 1, 2] and [5, 4]. Each half is recursively sorted. Then, the sorted halves are merged to obtain the final sorted array.

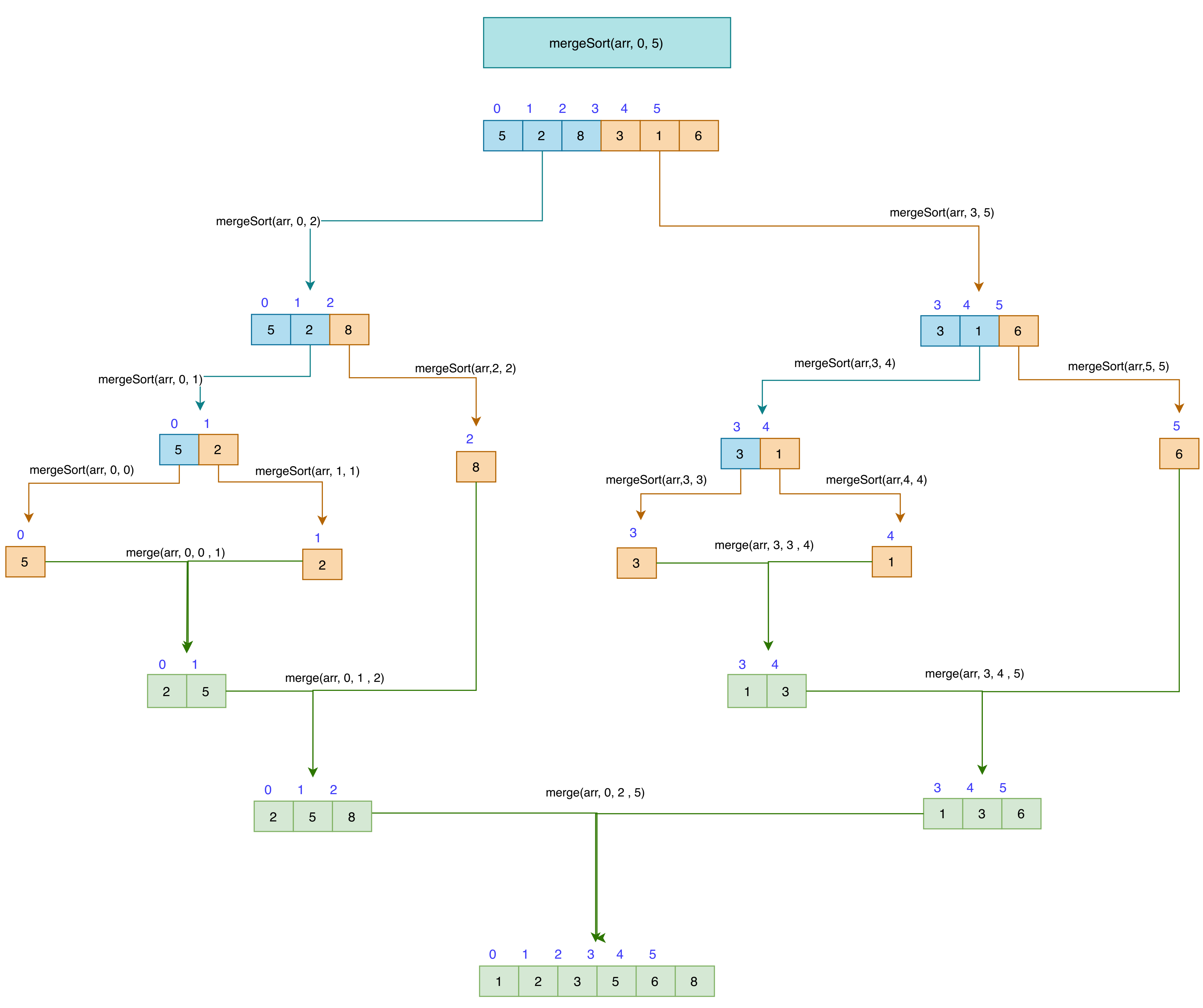
Solution

The algorithm follows these steps:

1. **Base Case:** If the input array has only one element or is empty, it is already considered sorted. Return the array.
2. **Divide:** Divide the input array into two equal halves.
3. **Recursive Call:** Recursively call the merge sort function on each half to sort them.
4. **Merge:** Merge the two sorted halves by comparing the elements and placing them in the correct order.
5. **Return:** Return the merged and sorted array.

The base case ensures that when the array is divided into single elements, they are considered sorted. The recursive case divides the array into smaller halves until the base case is reached. By recursively sorting and merging the halves, the entire array is sorted.

Let arr = [5, 2, 8, 3, 1, 6]



Code

Here is the code for this algorithm:

Java

```java
import java.util.Arrays;

public class Solution {
    public static int[] mergeSort(int[] arr) {
      mergeSortRecursive(arr, 0, arr.length - 1);
      return arr;
    }

    public static void mergeSortRecursive(int[] arr, int start, int end) {
        if (start < end) {
            int mid = (start + end) / 2;
            mergeSortRecursive(arr, start, mid); // Recursive call for the left half
            mergeSortRecursive(arr, mid + 1, end); // Recursive call for the right half
            merge(arr, start, mid, end); // Merge the sorted halves
        }
    }

    public static void merge(int[] arr, int start, int mid, int end) {
        int n1 = mid - start + 1;
        int n2 = end - mid;

```

Run Code Result Stdout

You must run your code first

Execute

Time and Space Complexity

The time complexity of the Merge Sort algorithm is $O(n * logn)$ in the average and worst case, where n is the number of elements in the input array. This is because the algorithm divides the array into halves recursively and merges them back together. The space complexity is $O(N)$ since it requires additional space for the temporary arrays during the merging process.

← Previous
12. Merge Sort

Next →
13. Quick Sort

☐ Mark as Completed